



StockPilot

Inventory Management System

Part 2 of 2 — **Codebase & File Guide**

What every important file does, and how the folders fit together.

Prepared 14 July 2026 · Author: Bhanu Prakash

o • The folder map

A single Next.js app — the API route handlers are the backend. Nothing touches the database or AI except code in `src/lib/`.

```
Inventory Management System/ (stockpilot)
├─ prisma/
│   ├─ schema.prisma      # tables + enums (PostgreSQL)
│   └─ seed.ts            # deterministic 90-day warehouse history
├─ src/
│   ├─ app/                # routes (folder = URL)
│   │   ├─ dashboard/ inventory/ movements/
│   │   ├─ purchase-orders/ suppliers/ advisor/
│   │   ├─ login/ signup/ page.tsx layout.tsx
│   │   └─ api/            # route handlers = backend
│   ├─ lib/                # the brain: db, auth, inventory, advisor
│   ├─ components/         # Shell, charts, per-page views
│   └─ types/
├─ .env / .env.example     # DB url, auth secrets, optional AI keys
└─ package.json / next.config.ts / tsconfig.json
```

1 • Data layer (prisma/)

DATABASE BLUEPRINT

prisma/schema.prisma

Declares the PostgreSQL datasource and every table. Models: `User` (bcrypt auth); `Supplier` (lead times); `Product` (SKU, cost/price, on-hand `quantity`, reorderPoint/Qty); `StockMovement` (the append-only ledger — `MovementType` IN/OUT/ADJUST, snapshot `balance`); `PurchaseOrder` + `PurchaseOrderItem` (`PoStatus` DRAFT/ORDERED/RECEIVED/CANCELLED); `AdvisorRun` + `ReorderSuggestion` (`Urgency` CRITICAL/SOON/OK/DEAD, `AdvisorSource` AI/HEURISTIC).

DATA GENERATOR

prisma/seed.ts

Deterministic PRNG (`mulberry32`, fixed seed) replays ~90 days of warehouse activity → ~30 SKUs, 6 suppliers, ~2,400 movements, 4 POs (2 received, 1 in-transit, 1 draft) + the demo user. Engineered stories: a fast mover stocked out with a PO in transit, SKUs below reorder point, dead-stock items — so the dashboard and advisor have a real narrative.

2 • The brain (src/lib/)

DATABASE CLIENT

src/lib/prisma.ts

One shared `PrismaClient` (global singleton) so dev hot-reload doesn't exhaust the connection pool.

QUERY HELPERS

src/lib/inventory.ts

All read queries + plain-JSON row DTOs: `getProducts` (with 30-day velocity & lastOut), `getMovements`, `getSuppliers`, `getPurchaseOrders`, `getWeeklyFlows`, and `stockStatus`.

THE AI ADVISOR

src/lib/advisor.ts

The reorder engine: `heuristicPlan`, `advisorPrompt`, `aiPlan` (Claude tool-use), `geminiPlan` (Gemini REST), `generateAndSavePlan` (the 3-tier cascade + persistence), `getLatestRun`.

AUTH + UTILS

src/lib/auth.ts • session.ts • utils.ts

NextAuth config (Credentials + JWT + bcrypt); `getSession()` server helper; formatters (`formatMoney`, compact money, date/relative-day labels).

3 · Routes, API & components

FILE	PURPOSE
<code>app/dashboard·inventory·movements·purchase-orders·suppliers·advisor /page.tsx</code>	Server components; each is <code>force-dynamic</code> , guards the session, reads via <code>lib/inventory</code> , and hands JSON to a client "View".
<code>app/api/products · products/[id]</code>	POST add SKU (+ opening-balance movement) · PATCH edit (not qty) / guarded DELETE.
<code>app/api/movements</code>	POST record IN/OUT/ADJUST — row-locked transaction, 409 on oversell.
<code>app/api/purchase-orders · [id]</code>	POST create draft PO · PATCH status machine (order / atomic receive / cancel).
<code>app/api/advisor</code>	POST run + persist the reorder plan.
<code>app/api/auth/[...nextauth] · register</code>	NextAuth handler · create operator account.
<code>components/Shell · Providers · Logo · Badges</code>	App chrome (sidebar nav), NextAuth SessionProvider, inline-SVG logo, icon+label status/urgency badges.
<code>components/charts/chartKit · dashboard/inventory/movements/purchase-orders/advisor Views</code>	Shared Recharts chrome + CVD-validated palette, and the interactive client view per page.

4 · Commands & the honest caveats

COMMAND	DOES
<code>npx prisma db push</code>	Create DB + tables from the schema (no migrations folder)
<code>npm run db:seed</code>	Replay the 90-day demo warehouse
<code>npm run dev</code>	Dev server → localhost:3000 · demo@stockpilot.app / demo1234

Honesty notes for the interview (code is ground truth): the README and `.env.example` label the DB **MySQL**, but the real `schema.prisma` and `.env` use **PostgreSQL** (proven by raw `SELECT ... FOR UPDATE` on double-quoted `"Product"`). The README credits only **Claude**, but the code adds a **Gemini** fallback tier; with the current `.env` (empty Anthropic key, populated Gemini key) a live run would use Gemini. The committed `.env` also contains real-looking secrets that would normally not be committed.